Testes em Express.js (MVC) com TypeScript

Instalação, configuração, anatomia e implementação

Por que testar

📉 **Diminuição de custo:**
Refatoração segura e menos bugs em produção.

⚡ **Feedback rápido:**
Saiba se quebrou algo em segundos, não em dias.

📖 **Documentação viva:**
O teste descreve como o sistema deve se comportar.

🏗️ **Pirâmide de Testes:**
Base sólida (Unitários) + Conectividade (Integração).

Instalação e Scripts

```
# Instalação das dependências de teste
npm i -D jest ts-jest @types/jest supertest @types/supertest
```

Ferramentas-chave:

- Jest: Runner e Framework
- ts-jest: Transformer para TypeScript
- Supertest: Testes HTTP (agente)

Scripts (package.json):

- "test": "jest"
- "test:watch": "jest --watch"
- "test:cov": "jest --coverage"

Configuração: jest.config.ts

```
// jest.config.ts
export default {
preset: 'ts-jest',
testEnvironment: 'node',
testMatch: ['**/*.spec.ts'],
clearMocks: true,
restoreMocks: true,
collectCoverageFrom: ['src/**/*.ts'],
coverageDirectory: 'coverage',
};
```

Destaques:

- **preset:** Ativa suporte TS nativo via ts-jest.
- **clearMocks:** Limpa chamadas de mocks entre testes.
- **testMatch:** Define o padrão de arquivos de teste (.spec.ts).

Estratégia: App vs Server

```
// src/app.ts
const app = express();
app.use(express.json());
app.use(routes);
export { app }; // Export sem listen
```

```
// src/server.ts
import { app } from './app';
app.listen(3000, () => {
console.log('Server running');
});
```

Por que separar?

1. **Velocidade:** Supertest inicia o app em memória instantaneamente.
2. **Isolamento:** Evita erros de "EADDRINUSE" (porta ocupada) no CI.
3. **Controle:** Permite injetar dependências ou mocks globais no app antes do teste.

Guia Prático: Setup & Scripts

```
# Scripts Recomendados (package.json)
"scripts": {
  "test": "jest",
  "test:watch": "jest --watchAll",
  "test:staged": "jest --bail --findRelatedTests",
  "test:cov": "jest --coverage",
  "test:int": "jest --config jest-integration.config.ts"
}
```

```
# Adicionar configuracao (tsconfig.json)
...
"compilerOptions": {
  "types": [“node”, “jest”],
}
...
```

Anatomia de um teste: O Padrão AAA

1. Arrange

Prepare o cenário: mocks, variáveis e instâncias.

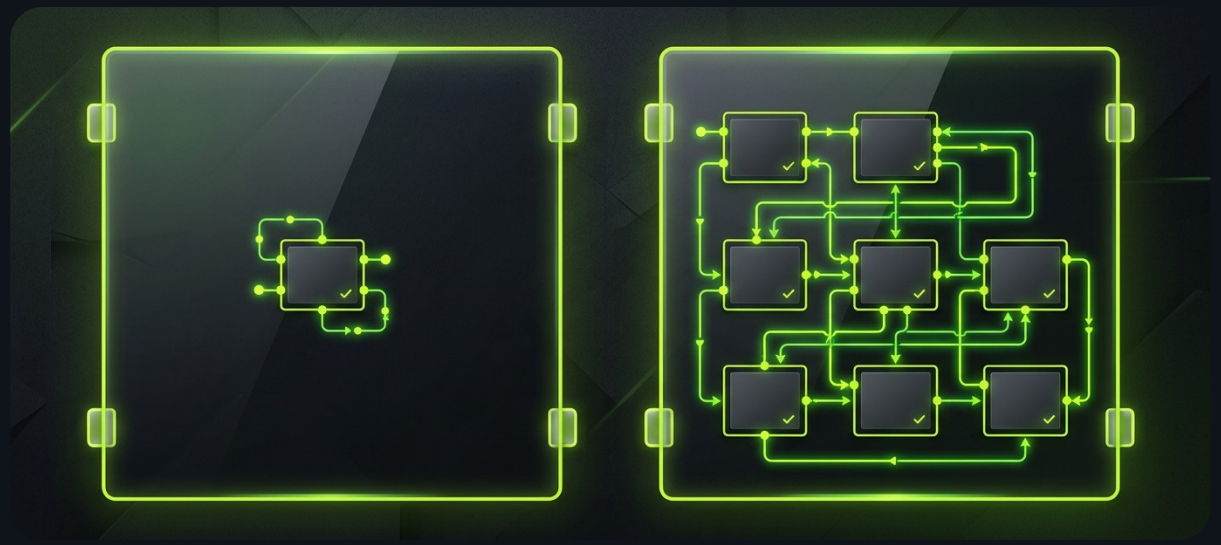
2. Act

Execute a função ou rota sob teste.

3. Assert

Verifique se o resultado foi o esperado.

Unitário vs Integração



Unitário:
Foca em 1 classe/função isolada. Usa mocks para dependências externas.

Integração:
Foca no fluxo entre camadas (Ex: Rota ➔ Controller ➔ Model).

Fator	Unitário	Integração
Velocidade	Ultra rápido (ms)	Mais lento (seg)
Dependências	Sempre Mockadas	Reais ou Stubbed

Exemplo Unitário: helpers

```
import {formatCurrency} from '../../../src/helpers/formatCurrency';

describe('formatCurrency', () => {
  it('should format 1.56 as R$ 1,56', () => {
// Arrange
    const value: number = 1.56;
// Act
    const received = formatCurrency(value);
// Assert
    expect().toEqual('R$ 1,56');
  });
});
```

Integração HTTP: Supertest

```
// Integração testa o fluxo real
import request from 'supertest';
import {app} from '../../app';

describe('GET /product', () => {
  it('should return success', async () => {
// Arrange, Act
    const response = await request(app).get('/product').send();
// Assert
    expect(response.status).toEqual(200);
    expect(response.body).toEqual([{brand: 'Trakinas', name:
'Bolacha'}]);
  });
});
```

Integração HTTP: Mock

```
import request from 'supertest';
import {app} from '../../app';
import * as productController from '../../../src/controllers/productController';

describe('GET /product', () => {
  it('should not return error, when empty value is returned', async () => {
// Arrange
    jest.spyOn(productController, 'getAll').mockReturnValueOnce([]);
// Act
    const response = await request(app).get('/product').send();
// Assert
    expect(response.status).toEqual(200);
    expect(response.body).toEqual([]);
  });
});
```

Cobertura de código

```
// package.json

...
"scripts": {
  ...
  "test:coverage": "jest --coverage"
},
...
```

Cobertura de código

- Métrica que indica quanto do código foi testado
- Expressa em percentual (%)
- Mostra quais partes foram executadas

npm run test:coverage

1. **Statement Coverage:** Mede quantas instruções/linhas foram executadas.
2. **Branch Coverage:** Mede caminhos de decisão (if/else).
3. **Function Coverage:** Mede quantas funções foram chamadas.
3. **Line Coverage:** Mede quantas linhas foram executadas.

Boas Práticas:
- cobertura >= 80%;
- Testar sucesso, erro e limites;
- Priorizar branches.